Runbook · Robot 3 cajas con el Unitree G1 (SONIC)

Versión limpia · 14/10/2026 · Robot real (G1 de 29 GDL) y simulador MuJoCo · se opera con el panel web.

El robot inspecciona, busca la caja roja, verde o azul, camina hacia ella y se detiene cuando su borde inferior llega a `y_max = 0.93` (igual para los tres colores). Se maneja con botones del panel, con teclado, por voz («zuu, busca el rojo») o con los mandos de los lentes Pico 4.

1. Qué hay y dónde corre

Pieza	Dónde	Función
`panel_g1.py` + `panel_g1.html`	PC del laboratorio (5062robotika, usuario `udirobotika`)	Panel web `http://127.0.0.1:8080`: arranca y detiene todo, muestra cámaras, botones, voz
`deploy_sonic_vision.py`	PC	Detecta colores, decide el movimiento y lo envía a SONIC por ZMQ `:5556`
`deploy.sh` (SONIC)	PC	Política de equilibrio y marcha; habla con el robot por `enp131s0` (o con el simulador)
`robot/start_camera.sh`, `robot/realsense_pub.py`	PC2 del robot (`192.168.123.164`)	Cámara RealSense: RGB (`:55555`) y, opcional, profundidad (`:55557`)
`mic_stream_g1.py`, `voz_cajas.py`	Robot → PC (UDP `:5600`)	Voz con el micrófono Insta360 del robot
`run_sim_3colores.py`, `sim_depth.py`	PC	Simulador MuJoCo con las 3 cajas y cámara (RGB `:5555`, profundidad `:5567`)
`depth_cam.py`	PC	Receptor/visor de profundidad (PNG 16 bits en mm, 0 = sin dato)
`pico_visor.py`	PC (entorno conda `tv`)	Lentes Pico 4: muestra lo que ve el robot y los mandos eligen el destino

Red: PC1 `192.168.123.161`, PC2 `192.168.123.164`, PC del laboratorio `192.168.123.222` (`enp131s0`).

2. Antes de empezar

1. Cierra las terminales manuales viejas (deploy, cámara, simulador): ocupan el puerto 5556 y la cámara.
2. Robot real: encendido, **de pie**, cable Ethernet conectado, espacio libre, control Unitree (o tecla `0`) a mano.
3. Revisa la red en la PC:

Comprobación	Debe verse
`ip -br addr show enp131s0`	`UP` y `192.168.123.222/24`
`ping -c 3 192.168.123.161` y `ping -c 3 192.168.123.164`	respuesta
`ip route get 192.168.123.164`	`dev enp131s0`

Si falla: revisa cable y que el robot haya terminado de arrancar (30–60 s). Sin IP: `sudo ip addr add 192.168.123.222/24 dev enp131s0 && sudo ip link set enp131s0 up`.

3. Traer la última versión y abrir el panel

```
cd ~/tres-colores && git pull          # si pide, antes: git stash
cd /home/udirobotika/DURVVIN/SIM2SIM/blea/GR00T-WholeBodyControl
source .venv_teleop/bin/activate
python ~/tres-colores/deploy/deploy_sonic/panel_g1.py
```

Abre `http://127.0.0.1:8080` y deja esa terminal abierta. La contraseña del robot y los cambios de código se aplican al reiniciar el panel (Ctrl+C y volver a lanzar).

4. Robot real, paso a paso

1. Modo **Robot real**.
2. Escribe la contraseña del robot y **Guardar en memoria** (vive solo en memoria; repetir tras cada reinicio del panel).
3. Primera vez o tras actualizar: **Preparar robot (scp)** (copia `mic_stream_g1.py`, `start_camera.sh`, `realsense_pub.py`).
4. «Servidor de cámara del robot»: **teleimager** (solo RGB, validado) o **RealSense propio** (RGB + profundidad; reinicia la cámara A).
5. Comprueba PC1, PC2 e Interfaz en verde y marca la casilla de seguridad.
6. **Arrancar todo**: cámara (A) → micrófono (M) → deploy (2) → programa de cajas (3). Confirma `SI` cuando lo pida; en el registro del deploy debe verse `Planner enabled`.
7. Prueba primero la marcha con **Prueba marcha** (`w`, 3 s), luego **Inspeccionar** (`s`) y **Rojo / Verde / Azul** (`1/2/3`).
8. Para detener: **PARAR** (ESPACIO) o **Detener todo**. Emergencia: **EMERGENCIA** del panel, tecla `0` en el deploy o control Unitree.

5. Simulador MuJoCo

1. Modo **Simulador MuJoCo**; marca «Simulador con profundidad» si quieres esa cámara.
2. **Arrancar todo**: simulador (1) → deploy (2) → programa de cajas (3). El simulador siempre va antes que el deploy.
3. En la ventana de MuJoCo suelta el robot: `7` dos veces y luego `9`.
4. Usa los mismos botones que en el robot real.
5. La ventana de MuJoCo se abre como una ventana normal de 1000×700, que se mueve con el mouse y se redimensiona desde el borde. Para cambiarla: Ajustes → «Ventana de MuJoCo» (`AnchoxAlto+X+Y`, por ejemplo `800x600+100+80`; `off` deja el tamaño de MuJoCo). A mano: `SIM_WIN=800x600+100+80 python .../run_sim_3colores.py`. El simulador se abre por XWayland (X11 dentro de la sesión Wayland), donde la ventana tiene barra; antes de forzarlo abre un visor de prueba de 1 s y, si el equipo no puede (`GLX: Failed to create context: BadValue` / `could not`

`create window`), usa la plataforma por defecto y lo avisa en el registro. `SIM_X11=0` lo desactiva a mano; sin `DISPLAY` (sin XWayland) no actúa.

6. Cámaras RGB y profundidad

La fila **Cámara** del panel tiene **RGB**, **Profundidad**, **RGB + profundidad** y **Ocultar**. La profundidad se ve en mapa de color con la distancia (m) al centro; el alcance se cambia en Ajustes (4 m por defecto).

	Robot real	**Simulador**
RGB	`head_camera` (`:55555`)	`head_camera` (`:5555`)
Profundidad	`realsense_pub.py` en el PC2 (`:55557`), opción «RealSense propio»	Casilla «Simulador con profundidad» (`:5567`), reiniciar el simulador

Visor suelto: `python depth_cam.py --host 192.168.123.164 --port 55557`.

7. Marcha y velocidad

- Solo se usan dos modos de SONIC: `IDLE` (quieto o giro en sitio, 0.3 rad/s) y `SLOW_WALK` (0.1–0.8 m/s).
- **Marcha continua:** con la caja poco desviada el robot camina sin parar y corrige el rumbo. Solo se detiene a girar en sitio si la desviación supera `--turn-tol` (0.65) y vuelve a caminar cuando baja de `--center-tol` (0.2). Si aún hay paradas, sube `--turn-tol` a 0.8.
- Valores por defecto: avance 0.3 m/s (tope real 0.3), acercamiento 0.15 m/s desde `y_max > 0.80`, parada en `y_max ≥ 0.93`.
- Subir velocidad por pasos (Ajustes → tope de velocidad real, o `--max-walk` hasta 0.8), probando cada paso con `w` y luego con la caja:

Paso	**Avance (m/s)**	**`--slow-y`**	**`--slow-speed`**
0 (actual)	0.3	0.80	0.15
1	0.4	0.75	0.20
2	0.5	0.70	0.20
3	0.6 o más	0.65	0.25

8. Voz

El micrófono Insta360 va por USB-C al robot: `mic_stream_g1.py` lo envía por UDP `:5600` a la PC y `voz_cajas.py` lo reconoce (Google, **requiere internet**). En el panel: activar micrófono (M) y «Aceptar órdenes de voz». Una vez en la PC: `uv pip install --python "$(which python)" SpeechRecognition`.

Frases: «zuu, busca el rojo / verde / azul», «inspecciona», «quieto». **«Para» o «alto» detienen el planner** (no sustituyen la parada de emergencia).

9. Lentes Pico 4 y mandos

Las gafas muestran **lo mismo que el panel** (el video del programa de cajas con su detección y estado; si ese no está, la cámara cruda que elijas con los botones RGB / Profundidad) y los mandos del Pico eligen el destino del robot. El panel sigue mostrando todo igual. El programa es `pico_visor.py`, que corre en el entorno conda `tv` y envía las mismas teclas que el panel.

Una sola vez: entorno conda `tv` con `televuer` (runbook del Pico: `runbook-pico4-camara-pc`), certificado en `~/.config/xr_teleoperate/{cert,key}.pem`, `sudo ufw allow 8012`, y el Pico en la misma red que la PC. En Ajustes, «Pico: IP de esta PC en la red del visor» (por defecto `192.168.123.104`; cambia por DHCP, verifícala con `ip -4 addr show`).

Cada vez: 1. Arranca todo como siempre (robot real o simulador + programa de cajas). 2. En el panel, tarjeta **Lentes Pico 4 y mandos** → **Iniciar visor** (o marca «Incluir en Arrancar todo»). Deben verse «Visor activo» y «Video del programa». 3. En el navegador del Pico (barra de direcciones, no el buscador): escribe `https://<IP>:8012/?ws=wss://<IP>:8012` (la página la sirve la propia PC), acepta el aviso del certificado («Avanzado» → «Continuar de todas formas») y pulsa **Virtual Reality**. El panel muestra «Gafas conectadas». 4. Vista **ego** (ventana con el entorno real alrededor, recomendada para trabajar con el robot) o **inmersiva**. Si solo ves la cuadrícula, gira y mira alrededor.

Mando	Orden
A (derecho)	Rojo
B (derecho)	Verde
X (izquierdo)	Azul
Y (izquierdo), mantener	Avanzar recto (repite `w`; al soltar: PARAR)
Gatillo derecho	**Elegir la caja a la que apuntas con el puntero**
Clic corto del joystick derecho	Inspeccionar (no pide grip: solo gira en el sitio)
Clic corto del joystick izquierdo	Quieto (no pide grip)
Gatillo izquierdo	**PARAR** (siempre funciona)
Gatillo derecho (puntero sin calibrar)	Calibrar el puntero apuntando a la cruz del centro
Botón 🎯 **Calibrar puntero** del panel	Cuenta atrás de 4 s en las gafas: apunta a la cruz del centro y se calibra solo
Los dos clics a la vez, o mantener un clic 1 s	Calibrar de nuevo el puntero
Grip (lateral)	Seguro

Puntero para elegir la caja. En las gafas, cada caja que el programa de cajas detecta lleva un recuadro fino (rojo, verde o azul). Apuntas con el mando derecho (un cursor amarillo muestra dónde cae el rayo), la caja apuntada se resalta con un recuadro grueso y su nombre, y el **gatillo derecho** manda al robot hacia ella, igual que pulsar su color. Pide el mismo seguro (grip) que las demás órdenes de movimiento. Solo funciona con el programa de cajas encendido (es quien detecta las cajas).

1. **Calibrar (cada vez que entras a las gafas):** apunta a la cruz amarilla del centro del video y aprieta los dos joysticks a la vez. El cursor aparece y el estado dice «puntero calibrado».
2. Apunta a una caja: se resalta. Aprieta el gatillo derecho: el robot va hacia ella y se detiene en `y_max = 0.93`.
3. Si el cursor no cae donde apuntas: «Plano del video» (fijo en el espacio / pegado a las gafas), «Pico: tamaño del plano del video» y el ajuste fino yaw/pitch están en Ajustes. Alternativa que no depende de la pose del mando: Puntero = **cursor con joystick izquierdo** (lo mueves con el joystick y eliges con el gatillo derecho).

Seguro (se elige en el panel): *habilitar* (por defecto): para dar un destino hay que tener un grip apretado en ese momento; *mantener*: además, si sueltas los dos grips el robot recibe PARAR; *sin grip*: no recomendado con el robot real. La barra inferior de las gafas muestra la leyenda de los botones, el estado del seguro y la última orden. La parada de emergencia real sigue siendo el mando Unitree o la tecla `0`.

Probar mandos (botón del panel): muestra qué botones detecta y **no envía nada al robot**. Úsalo la primera vez para confirmar que X/Y corresponden al mando izquierdo.

Síntoma	Solución
`no encuentro 'televuer'`	El panel lanza el visor con `conda activate tv`; cambia el entorno en Ajustes («Pico: entorno conda»)
Puerto 8012 en uso	Otra copia del visor o `test_pico_local.py` abierto: ciérralo
«Gafas sin conectar»	Abre la URL en el Pico y pulsa Virtual Reality; revisa la IP (Ajustes) y el certificado
Órdenes bloqueadas «mantén un grip apretado»	Aprieta un grip al pulsar (modo habilitar/mantener)
El gatillo dice «Apunta a una caja» o «Calibra el puntero»	Calibra (cruz del centro + 2 joysticks) y apunta dentro del recuadro de una caja; el programa de cajas debe estar encendido
El cursor no sigue al mando / se desvía	Cambia «Plano del video», ajusta el tamaño o el yaw/pitch; o usa el puntero con joystick
Las gafas dicen «Sin imagen del robot»	Inicia el programa de cajas o una cámara (RGB / Profundidad) en el panel

Estado: probado con gafas simuladas (mapeo de botones, envío de teclas al panel, estados en el panel, cierre ordenado). **Falta probar con el Pico real:** nombres de botones, posición del plano de imagen y precisión del puntero (la geometría es una suposición que se ajusta con la calibración y los ajustes del puntero; con gafas simuladas se verificó que el gatillo elige la caja correcta y bloquea sin grip, sin calibrar o sin caja), latencia y el visor completo con `televuer`.

10. Cerrar una prueba

1. **PARAR** (ESPACIO): robot quieto.
2. **Detener todo** en el panel (cierra programa de cajas, deploy, micrófono y cámara en orden).
3. Robot sentado o en arnés.
4. Antes de relanzar: `fuser 5556/tcp` vacío (si no, botón **Liberar puerto 5556**).

11. Problemas frecuentes

Síntoma	Solución
PC1, PC2 o Interfaz en rojo; `No route to host`	Revisar la red (sección 2)
El registro de A/M se queda sin avanzar	Guardar la contraseña del robot en el panel
`Device or resource busy` en la cámara	«¿Quién usa la cámara?»; `start_camera.sh` ya libera la cámara
`Address already in use` en 5556	**Liberar puerto 5556**
Deploy falla con `Address already in use` en 5557	Resuelto: la profundidad del simulador ahora usa 5567. Si tienes `panel_config.json` viejo, pon 5567 en Ajustes
Simulador marcado ERROR pero abierto	Resuelto en la última versión (el aviso «Channel factory init error» es inofensivo): `git pull` y reiniciar el panel
`Lost LowState` en el deploy	Se perdió la red con el robot: revisar sección 2 y reiniciar el deploy
«No llega profundidad»	Real: elegir «RealSense propio» y reiniciar A. Simulador: marcar la casilla y reiniciar el simulador
Pico: entra a Virtual Reality pero solo se ve la cuadrícula azul y negra; el panel dice «Gafas sin conectar»	El certificado de `https://<IP>:8012` no se aceptó (el websocket no conecta), o hay otra pestaña de Vuer abierta en la PC
Micrófono: `No encuentro ningun microfono`	No hay ninguna tarjeta de entrada: el receptor/micrófono no está en un puerto USB del computador del robot, o está apagado
Dice `LLEGO` sin caminar	La caja ya está más cerca que la parada; alejarla
El robot no camina	Ver `Planner enabled`, `avanzando mode=1` en el programa de cajas y `SLOW_WALK` en el deploy

12. Estado de la validación

Verificado: cámara RGB real (30 fps), detección de los 3 colores, panel completo con procesos simulados, contraseña/ssh, simulador con el panel, `Planner enabled`, micrófono → UDP → reconocimiento (prueba aislada), pipeline de profundidad con RealSense simulada.

Pendiente en el robot: confirmar la marcha continua tras el ajuste de histéresis, llegar a la caja y medir con cinta la distancia real de parada, voz integrada con el robot, profundidad con la RealSense real (`rgbd`) y con la escena completa del G1 en MuJoCo.